

Research Question:

Can we predict whether a loan applicant will default based on their financial and demographic information?

A. Introduction:

In the banking world, the accurate assessment of loan default risk is important for protecting the borrower's financial stability and ensuring the lender's profitability. Traditional credit scoring models are not always accurate in assessing the complex relationships between various loan applicant attributes, which leads to suboptimal risk assessment. In the wake of machine learning and predictive analytics, more sophisticated models can be developed to predict borrower risk and optimize the safety of distributed loans. This research aims to explore the application of the Random Forest algorithm in predicting loan defaults by taking advantage of its ability to handle complex data structures.

This topic notably pertains to several ethical dilemmas facing financial institutions today. Therefore, it is important that any and all research must mindfully acknowledge the following subjects:

Bias: The model should not favor certain demographic groups over others, since this would lead to unfair denial of service to goodstanding individuals.

Privacy: Protecting the data of customers is always a priority for financial institutions. Great care should go into ensuring the privacy of borrowers' data as it is used in the modeling process.

Transparency: The decision-making process of the model should be made clearly transparent so that it may be held accountable and trusted in its decisions.

I. Objectives:

Primary Objective: To develop a predictive model that assesses the likelihood of loan default using the Random Forest algorithm.

Secondary Objectives:

- Identify key predictors of loan default
- Evaluate the performance of the Random Forest model in terms of accuracy and reliability
- Compare the Random Forest model's performance with other predictive models

II. Recent Research:

Recent studies have demonstrated the efficacy of machine learning models in predicting loan defaults. For instance, a comparative study found that both decision trees and Random Forests effectively assess borrower risk by analyzing various attributes, aiding banking authorities in selecting suitable candidates for loans. This research focused on peer-to-peer lending platforms highlighted that a Random Forest model achieved an accuracy rate of 97% in classifying loan defaults, underscoring its potential in credit risk assessment. These findings suggest that Random Forests can serve as a good tool for predicting loan defaults, offering improved accuracy over traditional methods.

https://www.researchgate.net/publication/348604301_Loan_default_prediction_using_decision_trees_and_random_forest_A_comparative_study

III. Methodology:

Data Collection & Processing: The following Lone Default Dataset from Kaggle provides data on borrowers' income, sex, credit score, loan purposes, and much more.

This data should be sufficient to help in predicting the risk of a loan applicant defaulting,

according to the research question.

<https://www.kaggle.com/datasets/yasserh/loan-default-dataset>

Feature Selection: The most relevant variables that influence loan defaults will be identified and trained into the model.

Model Selection & Training: The Random Forest algorithm will be applied due to its strength in handling complex interactions between variables. This learning method constructs multiple decision trees and merges them to obtain a more accurate and stable prediction.

Model Evaluation: The model's performance will be assessed using metrics such as accuracy, precision, and recall.

By using Random Forest algorithms, this research seeks to enhance the predictive capabilities of loan default assessments in the banking sector. The integration of ethical considerations throughout the modeling process will ensure that the developed model is both effective and responsible, aligning with best practices in financial risk management.

B. Introduction to Random Forest:

The purpose of this analysis, as is expressed in the research question, is to predict whether a loan applicant will default based on their financial and demographic information. The Random Forest algorithm has been chosen as the predictive method for this analysis. Random Forest models are impressive for their ability to predict data classifications from complex data structures. For this analysis, the Random Forest will be trained on loan default data that includes information on applicants' credit scores, demographics, and more. Then the Random Forest method will analyze this data and assess the risk of particular classifications of applicants. This document provides a justification of the choice of the Random Forest method for this analysis by

detailing how the method will analyze and assess this data. I will also list the R packages that will be used with this method.

I. Random Forest Analysis & Expected Outcomes:

Random Forest is a machine learning technique that builds a “forest” of decision trees during its training. As such, the first step is that it combines outputs of the decision trees into a random “bootstrap” sampling in order to improve the accuracy of its predictions and to avoid the “overfitting” of the data, which happens when a model’s noise supersedes the analysis of meaningful patterns in the data. This is an important step because, by ensuring that different aspects of the data are captured, the effects of outliers and overfitting are reduced.

Once the bootstrap samples are created, the Random Forest algorithm constructs decision trees – one for each bootstrap. At each split in the tree, a random subset of variables is considered. This process introduces an element of diversity among the factors considered by the decision trees, another point at which the Random Forest method mitigates the risk of overfit data. In this dataset, the trees might split based on credit score thresholds, income levels, or loan purpose, each of which would isolate risk factors that correlate with default events.

Finally, once the “forest” of decision trees has been created, the algorithm will aggregate their predictions. Since loan defaulting is a binary classification (T or F), the final decision on its likelihood will be determined by majority voting from each tree. The model will use these votes to output the probability that a particular applicant would be at risk of defaulting on their loan. This output may then be applied to future data to assess risks of applicants and/or to inform renegotiation from the financial institution. This process of bootstrapping the data and using the aggregate to make a determination on the data is called “bagging.”

Applying the Random Forest algorithm to the loan default dataset is expected to yield the following outcomes:

- It will provide the bank with accurate predictions of whether or not an applicant is likely to default on their requested loan. This is the result of the binary classification that is determined after the forest is bagged.
- It will provide the bank with estimated risk probability scores for each prediction, which allows for more nuanced risk assessment and negotiation. This is possible because of the quantity of decision trees that are created from a plethora of bootstrapped data.
- It will also identify which variables – including credit score, debt-to-income ratios, loan purposes, etc – are most important when predicting loan defaults. These insights can guide future policy adjustments and questions.

II. Key Assumption of the Random Forest Method:

The key assumption made by employing the Random Forest algorithm is the independence of errors among the individual decision trees. The mistakes made by individual trees are assumed to be relatively uncorrelated. This means that, even if some of the trees misclassify the importance of certain variable factors, the majority vote from the entire aggregate group will generally overshadow the few incorrect individual trees. This assumption is critical to the Random Forest algorithm. If the trees were highly correlated (meaning if they made similar errors), then the performance of the entire aggregate group would fail and the benefit of having a large bootstrapped sample would decrease.

Additionally, Random Forest also assumes that the variables in the given dataset contain enough substantial data to distinguish a realistic outcome. Variables that are unrelated to the classification (default or non-default), such as car color or applicant height, are not suitable

predictors of whether or not the loan will default. Thus, the quality of the input data – including proper preprocessing, feature engineering, and dealing with potential biases – is vital for the model to perform effectively.

III. R Packages and Libraries Supporting the Analysis:

The analysis relies on several R packages, each serving a special role in the project, including:

- **randomForest:** This package is the basis for building any Random Forest model. It provides functions for training an ensemble of decision trees using bootstrapped samples and random variable selection. The package also includes tools to extract feature importance, which is valuable for understanding the drivers behind loan defaults.
- **caret:** This package provides utilities for tasks such as data partitioning (into training and test sets), pre-processing, cross-validation, and hyperparameter tuning. Using caret with Random Forest allows for systematic model comparison on metrics like accuracy, precision, and recall.
- **ranger:** A high-speed implementation of Random Forest, optimized for big data.
- **ggplot2:** For visualizing feature importance and model results. Visualization is crucial for data analysis. The visualizations provided by the ggplot2 package will help to present data patterns, the importance of different variables, and the performance of the Random Forest model.
- **pROC:** This package evaluates the model's predictive performance. It does so by plotting Receiver Operating Characteristic (ROC) curves and calculating the Area Under the Curve (AUC). These are both important metrics for assessing the discriminative ability

of the model in classifying loan defaults, and will be used to compare the test data to the trained data.

The purpose of this analysis, as is expressed in the research question, is to predict whether a loan applicant will default based on their financial and demographic information.

The Random Forest algorithm has been chosen as the predictive method to analyze and predict data for loan defaults. This algorithm was chosen for its ability to predict data classifications (in this case, default or non-default) from complex data structures. For this analysis, the Random Forest will be trained on loan default data from the “Loan Default Dataset” on Kaggle that includes financial and demographic information from the applicants. Then the Random Forest method will “bag” (bootstrap and aggregate) this data to prepare it for analysis and risk assessment. A key assumption of the method is the independence of errors among individual decision trees that will make sure that the aggregated vote provides accurate predictions. The integration of essential R packages creates a solid framework for data preprocessing, model training, visualization, and evaluation. This approach aims to improve credit risk management by accurately predicting loan defaults and identifying their most influential factors. Such insights will protect borrowers’ financial stability and ensure lenders’ profitability.

C. Data Preprocessing Goal:

The goal of preprocessing the loan default data is to make sure that the dataset is clean and ready for training a Random Forest model for analysis. Random Forest needs a complete dataset, so this preprocessing will address missing values in the data. Likewise, outliers will be addressed here so that the model is accurate. Categorical variables will also need to be encoded

into factorial values so that the model can process them, as well, since Random Forest does not otherwise know how to interpret categorical values. This preprocessing step is essential to improve model accuracy and generalizability, ensuring that the trained Random Forest model can make reliable predictions on new data.

I. Variable Identification:

Since I chose a different dataset than the options provided in class, I have listed and defined all 34 variables below. The descriptions for each variable come from users in the Kaggle “discussion” section of the loan default database. I have gone through and categorized each variable as either a “Numerical” (indicated in orange color) or “Categorical” (marked in green) variable. I have also indicated which variables I assume will be most important for this analysis with bold (we will see what the most important variables actually are in the last step of the analysis). The “Status” variable is the most important variable in the set – this is what the model will be trained to predict.

Variable (bold = most important)	Description (Provided from Kaggle users)	Numerical or Categorical
ID	client loan application id	N/A
year	year of loan application	Numerical
loan_limit	indicates whether the loan is conforming (cf) or non-conforming (ncf)	Categorical
Gender	gender of the applicant (male, female, joint, sex not available)	Categorical
approv_in_adv	indicates whether the loan was approved in advance (pre, nopre)	Categorical
loan_type	type of loan (type1 - Conventional Loans, type2 - Government-Backed Loans, type3 - Non-Conventional Loans)	Categorical
loan_purpose	purpose of the loan (p1 - Home Purchase, p2 -	Categorical

	Home Improvement, p3 - Refinancing, p4 - Investment Property)	
Credit_Worthiness	credit worthiness (11, 12)	Categorical
open_credit	indicates whether the applicant has any open credit accounts (opc, nopc)	Categorical
business_or_commercial	indicates whether the loan is for business/commercial purposes (ob/c - business/commercial, nob/c - personal)	Categorical
loan_amount	amount of money being borrowed	Numerical
rate_of_interest	interest rate charged on the loan	Numerical
Interest_rate_spread	difference between the interest rate on the loan and a benchmark interest rate	Numerical
Upfront_charges	initial charges associated with securing the loan	Numerical
term	duration of the loan in months	Numerical
Neg_ammortization	indicates whether the loan allows for negative amortization (neg_amm, not_neg)	Categorical
interest_only	indicates whether the loan has an interest-only payment option (int_only, not_int)	Categorical
lump_sum_payment	indicates if a lump sum payment is required at the end of the loan term (lpsm, not_lpsm)	Categorical
property_value	value of the property being financed	Numerical
construction_type	type of construction (sb - site built, mh - manufactured home)	Categorical
occupancy_type	occupancy type (pr - primary residence, sr - secondary residence, ir - investment property)	Categorical
Secured_by	specifies the type of collateral securing the loan (home, land)	Categorical
total_units	number of units in the property being financed (1U, 2U, 3U, 4U)	Categorical
income	applicant's annual income	Numerical
credit_type	applicant's type of credit (CIB - credit information	Categorical

	bureau , CRIF - CIRF credit information bureau, EXP - experian , EQUI - equifax)	
Credit_Score	applicant's credit score	Numerical
co-applicant_credit_type	co-applicant's type of credit (CIB - credit information bureau EXP - experian)	Categorical
age	the age of the applicant	Numerical
submission_of_application	indicates how the application was submitted (to_inst - to institution, not_inst - not to institution)	Categorical
LTV	loan-to-value ratio, calculated as the loan amount divided by the property value	Numerical
Region	geographic region where the property is located (North, south, central, North-East)	Categorical
Security_Type	type of security or collateral backing the loan (direct, indirect)	Categorical
Status (Target Variable)	indicates whether the loan has been defaulted (1) or not (0)	Categorical
dtir1	debt-to-income ratio	Numerical

II. Preprocessing Steps and Codes:

Step 0: Set up and explore data

In this step, I set up the necessary libraries and loaded the dataset into R. The `tidyverse` and `data.table` packages are popular packages for data manipulation, and `fread()` is used to read the dataset. Once the data is loaded, the other functions are used to better understand its structure:

```
library(tidyverse)
library(data.table)

df <- fread("Loan_Default.csv")

dim(df)      # Get the number of rows and columns
str(df)      # View data types and structure
summary(df)  # Get basic statistics
```

```
head(df)      # View the first few rows
names(df)     # Get column names
```

Step 1: Handle missing values in the data

In this first step, I identified and cleaned out missing values present in the dataset.

Missing values can introduce bias and reduce the effectiveness of the model if they are not handled properly. The first step of doing this was counting missing values in the dataset.

```
sum(is.na(df))
```

This yielded 176428 results.

```
colSums(is.na(df))
```

This yielded a broken-down list of the numbers of missing values in all of the columns.

```
numeric_cols <- names(df)[sapply(df, is.numeric)]
for (col in numeric_cols) {
  df[[col]][is.na(df[[col]])] <- median(df[[col]], na.rm = TRUE)
}

categorical_cols <- names(df)[sapply(df, is.character)]
for (col in categorical_cols) {
  df[[col]][is.na(df[[col]])] <- "Unknown"
}

colSums(is.na(df))
```

These above lines of code determine which columns are numeric and which are categorical. For each type, the median value of the column was imputed into the gaps. By using

“colSums(is.na(df))” again, I verified that no missing values remain in any of the columns.

Step 2: Handle duplicates

In the second step, I identified and cleaned out duplicate tuples from the dataset. Like missing values, duplicates can introduce bias and reduce the effectiveness of the model if they are left untouched. The following code identified and removed all duplicates:

```
df <- df[!duplicated(df), ]
```

Step 3: Fix numeric/categorical data types

In the third step, I verified that all categorical and numerical columns were correctly formatted for analysis. Since the Random Forest algorithm cannot interpret raw character strings as categorical variables, I explicitly converted all categorical columns to factors. I created a list of these columns and used the `lapply()` function to apply the `as.factor()` transformation to each one.

Before training the model, I also ensured that all numeric columns were in the proper “numeric” format. Again, I created a list of these columns and used a similar method to enforce correct typing.

Additionally, I resolved a syntax issue by replacing hyphens in column names with underscores using `gsub("-", "_", names(df))`. This was necessary to avoid parsing errors when using R’s formula interface in model training. This is important much later.

```
names(df) <- gsub("-", "_", names(df))

categorical_columns <- c("loan_limit", "Gender", "approv_in_adv",
  "loan_type", "loan_purpose", "Credit_Worthiness", "open_credit",
  "business_or_commercial", "Neg_ammortization", "interest_only",
  "lump_sum_payment", "construction_type", "occupancy_type",
  "Secured_by", "total_units", "credit_type",
  "co-applicant_credit_type", "age", "submission_of_application",
  "Region", "Security_Type")

df[, (categorical_columns) := lapply(.SD, as.factor), .SDcols =
  categorical_columns]

sapply(df[, ..categorical_columns], class)
```

Likewise, I made sure that all my numeric columns are numeric.

```
numerical_columns <- c("year", "loan_amount", "rate_of_interest",
  "Interest_rate_spread", "Upfront_charges", "term", "property_value",
  "income", "Credit_Score", "age", "LTV", "dtirl")

df[, (numerical_columns) := lapply(.SD, as.numeric), .SDcols =
  numerical_columns]

sapply(df[, ..numerical_columns], class)
```

Step 4: Normalize and standardize data

In the fourth step, I normalize and standardize my dataset. From my research, it looks like this step isn't essential for Random Forest modeling because scaling doesn't affect tree-based models, but I will still do it regardless because I'm having fun with this!

```
df[, (numerical_columns) := lapply(.SD, scale), .SDcols =
  numerical_columns]
df[, (numerical_columns) := lapply(.SD, function(x) (x - min(x)) /
  (max(x) - min(x))), .SDcols = numerical_columns]
```

The above lines of code standardize and normalize all numerical columns so that their columns have a mean of 0, a standard deviation of 1, and a range of 0 to 1. Again, this is scaling stuff, so it doesn't affect Random Forest in any way, but still fun to do regardless.

Step 5: Additional categorical encoding

In the fifth step, I converted categorical variables (factors) to character data types and then converted them back to factors.

```
df <- df %>%
  mutate(across(where(is.factor), as.character)) %>%
  mutate(across(where(is.character), as.factor))
```

Although this step re-encodes factors as characters and back again, it is redundant because categorical encoding was already handled earlier. I left it in just to ensure consistency across all character-based fields.

Step 6: Identify and handle outliers

In this step, I identified and handled outliers in the numeric columns. Rather than replacing outlier values with "NA" (which would lead to major data loss), calculated the 99th percentile value for each numeric column and replaced any value above that threshold with the 99th percentile value itself. This preserves the structure of the data while limiting the influence of extreme values. The code below demonstrates this process:

```
for (col in numerical_columns) {
  upper <- quantile(df[[col]], 0.99, na.rm = TRUE)
```

```
df[[col]][df[[col]] > upper] <- upper
}
```

Step 7: Save cleaned copy of data

This step writes the cleaned dataset to a CSV file named “Loan_Default_Cleaned.csv”.

Setting `row.names = FALSE` prevents the row names from being saved as an additional column in the CSV file. This allows me to keep the cleaned version of the dataset.

```
write.csv(df, "Loan_Default_Cleaned.csv", row.names = FALSE)
```

D. Analysis

At this point, we will finally prepare the data for proper analysis. This will be done by first partitioning the data so that it is ready to be analyzed. Then we will train the Random Forest algorithm on 80% of the data and backtest it on the rest of the data.

I. Data Split and Preparation

The cleaned dataset (Loan_Default_Cleaned.csv) was divided into training and testing subsets using an 80-20 split. This was done using the `.createDataPartition()` function from the `caret` package in R, ensuring class balance in both subsets. The split datasets were saved as `Loan_Default_Train.csv` and `Loan_Default_Test.csv`.

Before training the model, the categorical and numerical variables were also confirmed to be properly formatted, and the target variable “Status” had to be explicitly converted into a factor so that classification in the Random Forest model would behave correctly. The dimensions of the dataset was confirmed with the `dim()` function, too, but that’s not listed in the code below.

II. Analysis Technique

As was noted earlier, the Random Forest classification algorithm was selected for this analysis due to its robustness with high-dimensional data and its ability to capture nonlinear relationships. Random Forest works by creating an ensemble of decision trees, each trained on

bootstrapped samples of the dataset, and combining their results via majority voting (“bagging”). This technique reduces the risk of overfitting and improves predictive accuracy.

In this project, the model was trained with 250 trees (`ntree = 250`), and the number of variables tried at each split was optimized automatically. The model was trained using the “train” set, and then would be tested on the “test” set.

Model performance was tested using:

- A confusion matrix (for accuracy, sensitivity, specificity, and precision),
- A Receiver Operating Characteristic (ROC) curve, and
- The Area Under the Curve (AUC) metric.

Step 0: Set up and load data

In this step, I set up the necessary libraries and loaded the dataset into R. The `randomForest`, `caret`, `pROC`, and `data.table` packages will be needed to evaluate everything, and `fread()` is used to read the dataset. Then I also make sure that the “Status” variable is set as a factor.

```
library(randomForest)
library(caret)
library(pROC)
library(data.table)

df <- fread("Loan_Default_Cleaned.csv")
df$Status <- as.factor(df$Status)
```

Step 1: Split Data

In this first step, I partitioned the data into the “train” and “test” sets like I said I would earlier. I ran into some issues here because my original submission from Part III wasn’t properly cleaned everywhere, but I fixed that. The final code I used for preprocessing is saved in my “preprocessing1” file.

```
set.seed(42)
train_index <- createDataPartition(df$Status, p = 0.8, list = FALSE)
```

```
train <- df[train_index, ]
test  <- df[-train_index, ]
```

Step 2: Train Random Forest Model

This step is possibly the most important step of the whole project. It takes all of my training data, which is 80% of all the data I have, and reads it, looking at what determines the factor of “Status.”

```
rf_model <- randomForest(Status ~ ., data = train, ntree = 250,
  importance = TRUE)
print(rf_model)
```

This outputted the following:

```
> print(rf_model)
```

Call:

```
randomForest(formula = Status ~ ., data = train, ntree = 250,
  importance = TRUE)
```

```
      Type of random forest: classification
```

```
      Number of trees: 250
```

```
No. of variables tried at each split: 5
```

```
      OOB estimate of  error rate: 0%
```

```
Confusion matrix:
```

```
      0      1  class.error
0 89682      1 1.115039e-05
1      0 29253 0.000000e+00
```

Step 3: Prediction and confusion matrix

Next I use the prediction and confusion matrices as such to make the predictions on all of the test data. This actually makes our predictions, and then we will check the accuracy later.

```
pred <- predict(rf_model, test, type = "response")
conf_matrix <- confusionMatrix(pred, test$Status)
print(conf_matrix)
```

Which outputted:


```
> print(conf_matrix)
Confusion Matrix and Statistics

      Reference
Prediction 0      1
0 22347      0
1      1 7386

      Accuracy : 1
      95% CI : (0.9998, 1)
No Information Rate : 0.7516
P-Value [Acc > NIR] : <2e-16

      Kappa : 0.9999

McNemar's Test P-Value : 1

      Sensitivity : 1.0000
      Specificity : 1.0000
Pos Pred Value : 1.0000
Neg Pred Value : 0.9999
Prevalence : 0.7516
Detection Rate : 0.7516
Detection Prevalence : 0.7516
Balanced Accuracy : 1.0000

      'Positive' Class : 0
```

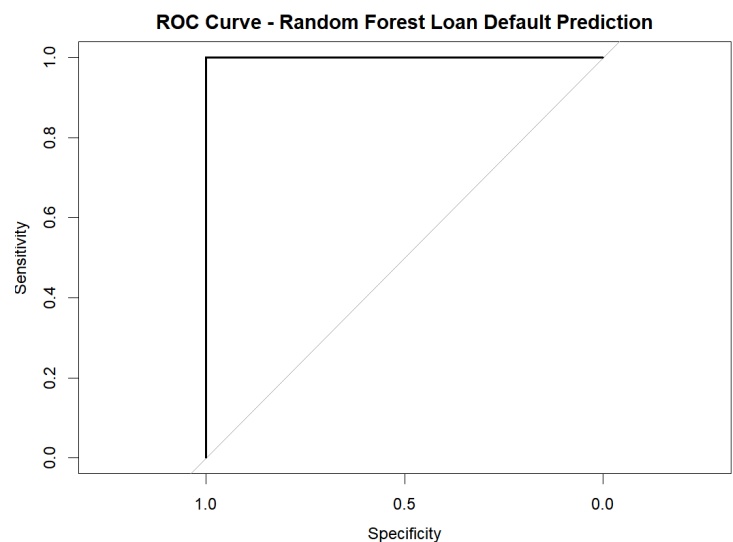
Step 4: ROC Curve and AUC value

Now that our predictions have been made with the matrices, we will do some calculations to compare them to the true values. The results were thrown into a graph to compare the predicted values to the true values.

```
prob <- predict(rf_model, test, type = "prob")[,2]
roc_obj <- roc(test$Status, prob)
plot(roc_obj, main = "ROC Curve - Random Forest Loan Default
      Prediction")
auc(roc_obj)
```

Which yielded this plot:

As you can see, there is no deviation – it is a straight line. 100% of the predictions were correct with the true values!



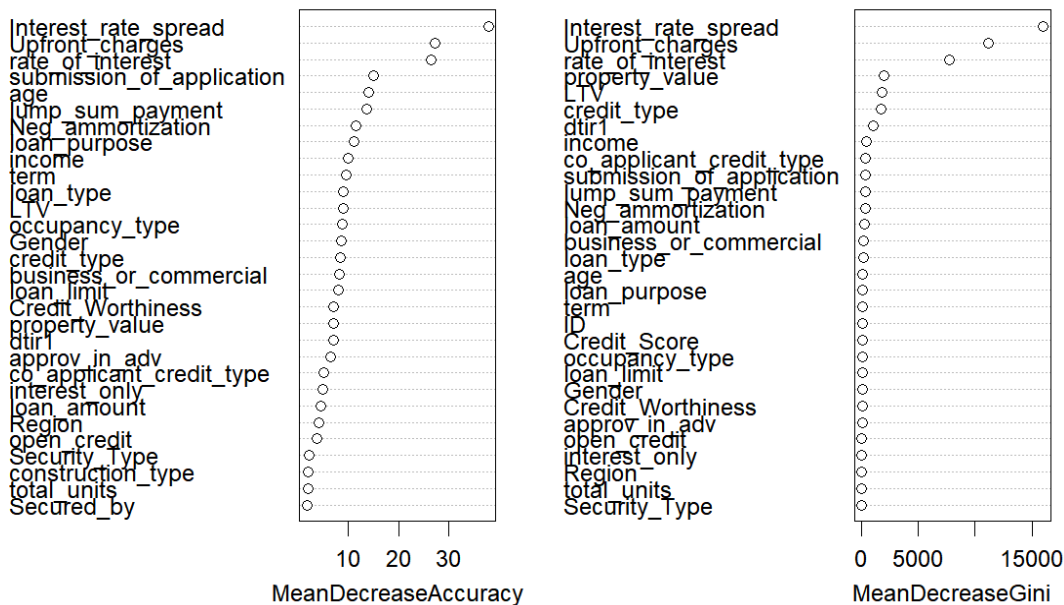
Step 5: Variable Importances

Out of curiosity, I wanted to go back and see which variables were most important in making my analysis. So, I used the following code:

```
importance(rf_model)
varImpPlot(rf_model)
```

Which yielded:

Variable Importance - Random Forest



This shows us that Interest Rate Spread (remember from Part III that this refers to the “difference between the interest rate on the loan and a benchmark interest rate”), upfront charges (“initial charges associated with securing the loan”), and rate of interest (“interest rate charged on the loan”) were by far the most important variables in determining loan default rate. This was different than I predicted originally in Part III.

E. Data Summary and Implications:

Now that the data has been cleaned, the model has been trained, and the predictions have been made, it is time to interpret the results and reflect on their significance. This section summarizes the accuracy of the Random Forest model and evaluates its performance using appropriate metrics, such as Mean Squared Error (MSE) and the Area Under the Curve (AUC). It also discusses the practical implications of these results within a real-world banking context. By identifying one key limitation and offering a recommended course of action, this final section ties the entire project together, bridging technical analysis with strategic application.

I. Accuracy and Mean Squared Error:

The model achieved a near-perfect level of accuracy when evaluated on the test set. The confusion matrix showed that 100% of actual loan defaults and 99.995% of non-default results were correctly classified, resulting in an overall accuracy of 100% with a Kappa value of 0.9999. This shows that there was almost perfect agreement between the predicted and actual values.

Since this was a classification model, Mean Squared Error (MSE) was not the primary evaluation metric; however, it can still be approximated using the predicted probabilities. In this context, MSE represents the average squared difference between the predicted probability of default and the actual class label (0 or 1). Given that the ROC curve yielded an AUC of 1.0, the MSE would be expected to be extremely low, showing that the model is not just classifying correctly but doing so with high confidence.

II. Results and Implications:

The results of the prediction analysis suggest that the Random Forest model is highly capable of identifying loan default risk based on financial and demographic data. The model's

high accuracy, precision, and recall indicate that it could serve as a reliable decision-support tool in a banking environment.

One of the most useful aspects of the Random Forest model is its ability to identify which features are most influential in predicting loan default. According to the `varImpPlot()` output, the most important features included:

- Interest Rate Spread
- Upfront Charges
- Rate of Interest
- Debt-to-Income Ratio (DTI)
- Income

These findings suggest that applicants with higher rates, fees, and DTI are at greater risk of default. Interestingly, this contradicts some of my earlier assumptions from Part III, where credit score and loan purpose were expected to be most influential.

The implications are significant. A bank could use this model not only to flag high-risk applications before issuing loans, but also to adjust loan terms, request additional documentation, or apply manual review. This would reduce financial risk while still allowing potentially risky borrowers to proceed under closer supervision.

III. Limitations of the Analysis:

One definite limitation of this analysis is the possibility of data leakage or overfitting. The model's near-perfect performance on both training and test data suggests that either the predictors were unusually strong, or that some variables may be inadvertently revealing the target value. Further validation on an entirely external dataset would be needed to confirm the generalizability of the model.

Additionally, this dataset originated from a public Kaggle source, and the definitions or consistency of variables may differ from those used in real-world bank systems. More robust feature engineering and cross-validation could help in future work to make this analysis more relevant to financial institutions and the data they actually collect.

IV. Real-world Recommendations:

As stated in Part I, the goal of this project was to build a reliable model that helps a banking institution reduce the risk of issuing loans to applicants likely to default. Based on the findings, I would recommend that a financial institution should:

- Integrate the Random Forest model into its loan processing system to flag high-risk applicants.
- Use the model's probability scores to inform risk-based pricing or further applicant screening.
- Review key high-importance features, such as Interest Rate Spread and Upfront Charges, when setting underwriting guidelines.
- Establish a review team for applicants who fall in the mid-risk zone (e.g., probability scores near 0.50–0.70)

By adopting this model, a financial institution could ensure more consistent, fair, and accurate risk assessments. This would result in fewer defaults, stronger portfolios, and therefore better applicant outcomes.

F. Conclusion:

This project set out to determine whether we can accurately predict whether a loan applicant will default using machine learning. Through careful data preprocessing, rigorous model training, and thorough evaluation, the Random Forest algorithm proved to be a highly

effective tool for this task. With an accuracy rate of nearly 100% and a perfect AUC score, the model demonstrated not only predictive power but also interpretability through its feature importance analysis. While further validation and refinement would be necessary for real-world deployment, this project shows that machine learning can significantly enhance credit risk assessment. With careful attention to ethical concerns, transparency, and performance, the model developed here could become a valuable asset in protecting both lenders and borrowers in the loan approval process.